

DeepL API – HTML Handling

E-commerce Localisation Demo

Technical Solutions Manager – Case Study

Lilla Csanaky

May 2026

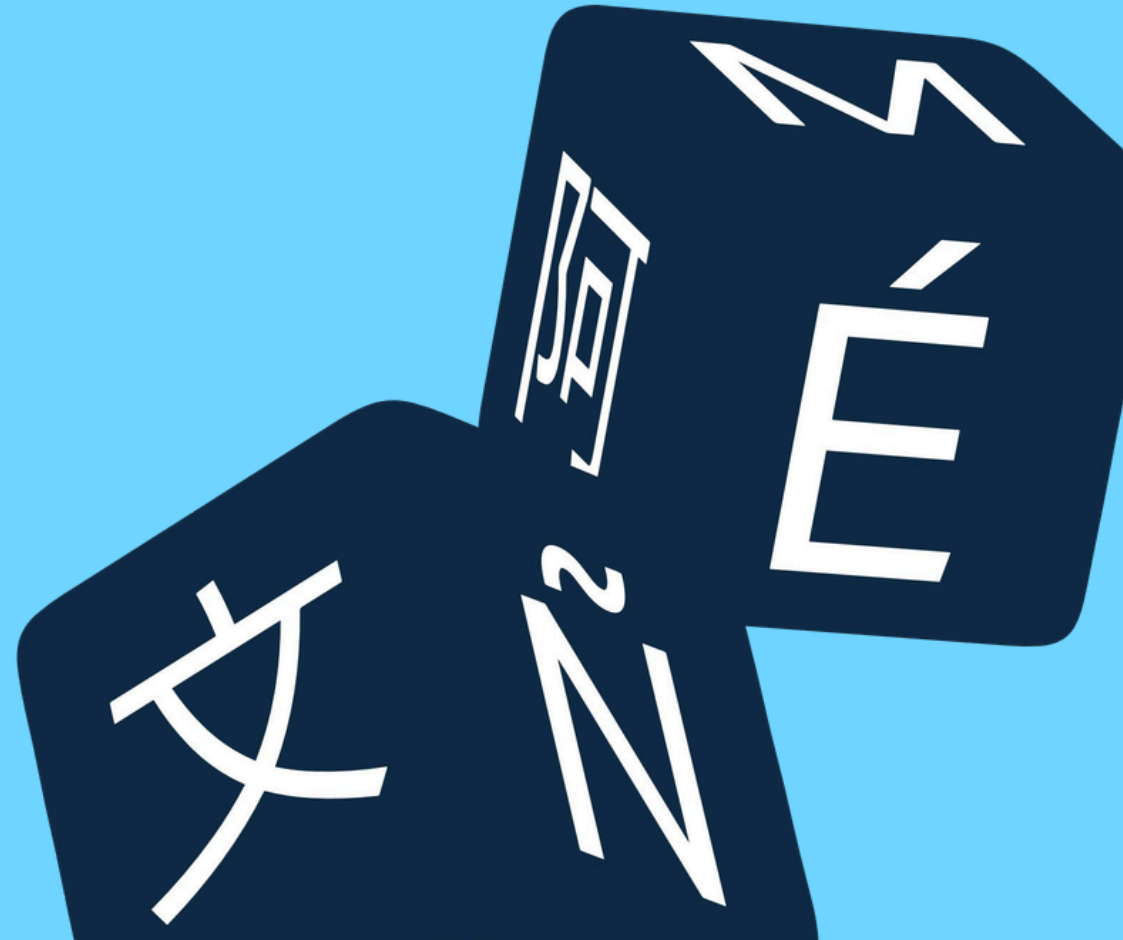


Professional Services
Customer Team



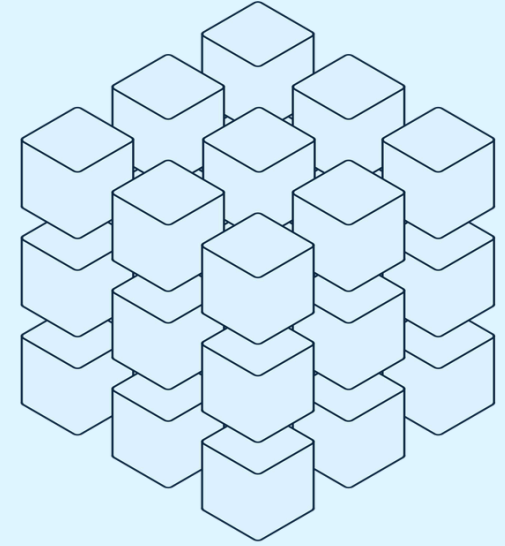
Agenda

1 Scenario and assumptions	5 min
2 Questions I would ask	5 min
3 API usage patterns	10 min
4 Live demo	20 min
5 Pricing and ROI	5 min
6 Documentation feedback	5 min
7 Q&A	10 min



The scenario

Customer	Existing DeepL Pro Advance license E-commerce webshop
Today	Individuals doing static, manual translation inputs via the DeepL app or web interface
Goal	Switch to the DeepL API to programmatically localise their website and expand to new markets
Focus	DeepL API — HTML handling feature Technical viability, Implementation approach, ARR expansion
Audience	Software engineers + technically savvy product manager



Assumptions

About the account

- I have not worked with this customer directly before
- I am familiar with the account and have reviewed their webshop
- The CSM has already introduced the API concept – I am picking up from there on the technical side

About the conversation

- This is a technical feasibility conversation, not a sales call
- The engineering team has been asked to evaluate viability
- The business decision follows from their technical sign-off

About the customer

- They are comfortable with DeepL – no product introduction needed
- HTML handling is the specific feature they want to understand
- They are open to expanding their usage if the integration is viable
- I am working in partnership with the CSM on this account

Questions I would ask

Content & catalogue

- How many products are in the catalogue today?
- How many new products or updates per day or per week?
- Which languages are the priority markets?

Team

- Do you have backend engineers available for this integration?
- Who would own the ongoing maintenance?

Technical setup

- What is your current stack — custom build, Shopify, headless CMS?
- Is content stored in a database or a CMS, and does it have API or webhook support?
- Do you have a CI/CD pipeline where a translation step could be inserted?
- Who owns the integration — backend, frontend, or both?
- What are your page load latency requirements?

Why these matter

The answers determine which integration pattern fits — build-time, runtime, or delta — and what the implementation effort looks like.

API essentials

Endpoint	POST /v2/translate
Authentication	Authorization: DeepL-Auth-Key YOUR_KEY Free tier: api-free.deepl.com Pro tier: api.deepl.com
Billing	Charged per character of source text — not per API call Tags, attributes, and markup are not billed Response includes billed_characters per request

Key parameters	tag_handling=html tag_handling_version=v2 (default for new customers) source_lang target_lang formality=less / more (prefer_less / prefer_more) glossary_id split_sentences=nonewlines
Quick start	pip install deepl client = deepl.DeepLClient(api_key) result = client.translate_text(html, target_lang="DE", tag_handling="html", tag_handling_version="v2")

API usage patterns (1/2)



Build-time translation

Content is translated once and stored.
API is called when content changes — not when users browse.
EN content updated → API call → translated file stored → served statically

Pro: Fast page loads, predictable API costs, no runtime dependency

Con: Translated content is only as fresh as the last run

Real: CMS webhook triggers translation script on publish

✓ Recommended for e-commerce



Delta translation

Only changed content is sent to the API. A cache tracks what has already been translated.
content changed? → yes → translate + update cache → no → skip, 0 characters billed

Pro: Minimal API usage, cost scales with changes not catalogue size

Con: Requires change tracking logic

Real: Run daily or on publish · compare against stored hash or timestamp

✓ Efficient at scale



Runtime translation

API is called live on every page request.

Pro: Always up to date , simple to implement

Con: Latency on every load, cost scales with traffic not content

Real: Only viable for low-traffic pages or as a fallback

⚠ Use with caution

API usage patterns (2/2)



Translation memory

Previously translated segments are stored and reused.

Identical phrases across products are translated once.

"Free returns within 30 days" → translated once → reused across all products

Pro: Significant cost reduction for large catalogues with repeated content

Con: Requires a translation memory system or TMS integration

Real: DeepL supports Translation Memories — upload via API or dashboard

✓ Reduces cost at scale



Runtime as fallback only

Pre-translated content is served statically. Runtime translation handles edge cases only — new products not yet translated, cache misses, new markets just opened. Result is cached immediately after first translation.

Pro: Fast for most users, resilient for edge cases

Con: More complex architecture

Real: Production pattern for large-scale e-commerce

✓ Best of both worlds



Real-world architecture

Content created/updated (EN)



Translation script triggered (build-time or delta)



Result stored in CMS / database / static file



Page served in target language — no API call at load time



Runtime fallback for untranslated content → cached on first hit

Live Demo

Three steps – increasing complexity

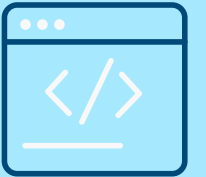
1 Postman

Raw API call – request and response `tag_handling=html,`
`tag_handling_version=v2`



2 demo.py

Same call in Python – 15 lines of code DeepL Python
library, HTML in, translated HTML out



3 Example: e-commerce demo

Fictional luxury bath bomb webshop
Build-time / Delta / Runtime translation
Glossary, Character billing
Edge cases



Pricing and ROI

The cost of manual translation

- How many people spend time on translation today?
- How many hours per week?
- How often does product content change?

Example

1 person, 10 hours/week, €50/hour = €25,000/year in translation labour
Not counting errors, inconsistency, or delayed market entry

Manual	API
Scales with headcount	Scales with content
Inconsistent output	Consistent, auditable
Slow to market	Translate on publish

Accuracy and billing

Bad translation has a hard cost

- Returns due to misunderstood product descriptions
- Lost conversions and brand damage in new markets
- "Sneaker" becoming "Turnschuh" across half the catalogue



Glossary support fixes this
Brand terms always translate correctly

How API billing works

- Charged per source character — not per call, not per word
- Tags and markup are free — only text inside counts
- Enterprise pricing is volume-based and negotiated

Explaining HTML handling to a non-technical stakeholder

The analogy

HTML is like a set of labelled boxes. Each box has a type (heading, paragraph, button) and contains text. DeepL opens each box, translates what's inside, and puts it back. The boxes themselves (their labels, their links, their structure) stay exactly as they were. Only the words change.

What this means in practice

Without HTML handling

The developer has to strip the formatting before translating, then manually rebuild it afterwards. Error-prone, time-consuming, hard to automate at scale.

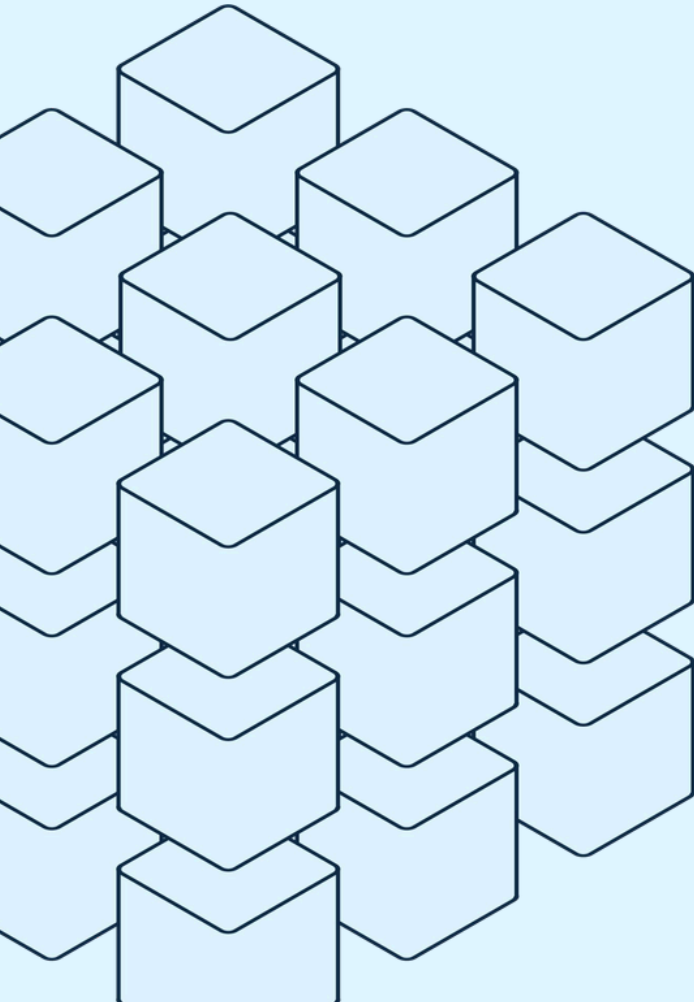
With HTML handling

The formatted content goes in. Translated formatted content comes out. No stripping. No rebuilding. The page works as-is in the new language.

The business value

- Faster time to market in new languages
- No manual formatting work after translation
- Consistent output across the entire product catalogue
- One API call per page – not one per sentence

API documentation feedback



What works well

- v2 page is clearly structured
- HTML example is relevant and easy to follow
- Migration guide from v1 is useful for existing customers

Gaps and suggestions

1. **alt, title, placeholder attributes**

Not translated in HTML mode — not documented

Workaround: extract, translate as plain text, reinsert

2. **script and style tags**

Should be skipped automatically in HTML mode

Currently not reliable in v2 — content may be encoded

Workaround: externalise scripts before sending HTML to the API

3. **ignore_tags parameter**

Only works in XML mode — not available for HTML mode

Not clearly stated in the docs

4. **Glossary immutability**

Glossaries cannot be updated — delete and recreate required ones

Not documented explicitly

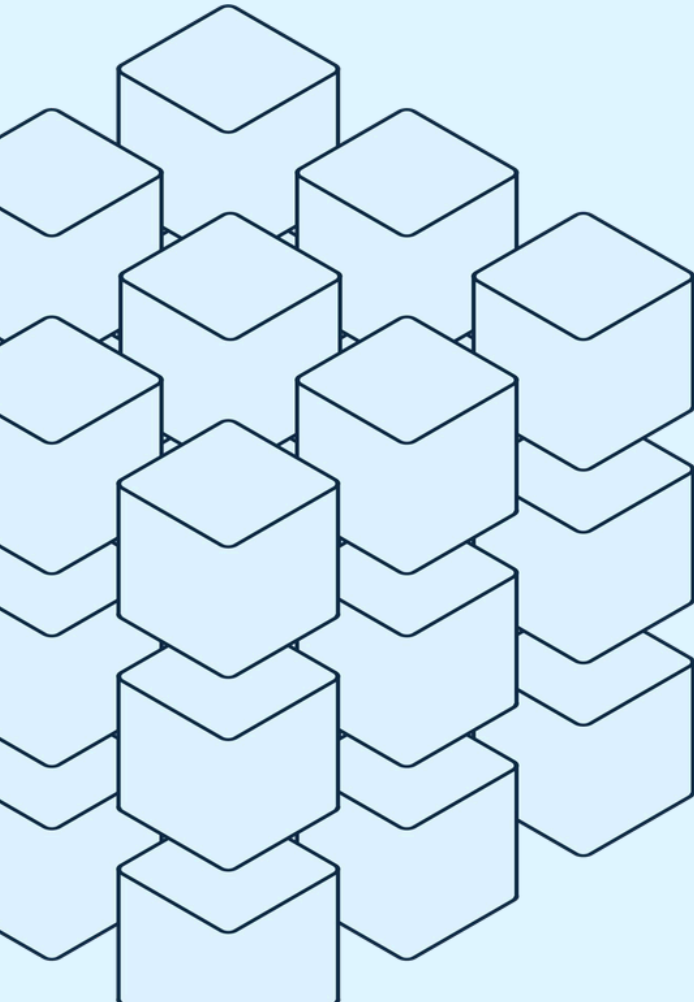
source_lang is required but not marked as mandatory in some examples

5. **Missing content**

No example with mixed real-world content (text, image, link, button)

No getting started path for new API users coming fresh to tag handling

Questions the customer might ask



Technical

- Does the API preserve our CSS classes and data attributes?
- What happens if we send invalid or malformed HTML?
- How do we handle alt text and image descriptions?
- Can we translate multiple pages in one API call?
- What is the request size limit?
- How do we handle content that should never be translated — brand names, product codes, SKUs?
- Can we use the API with our existing i18n framework?
- What happens if the API is down — is there a fallback?

Commercial

- How is pricing calculated at our volume?
- How does this affect our current Pro Advance contract?
- What SLAs are available for the API?

Implementation

How long would a basic integration take?

Do you have examples for our specific stack?

Who supports us during implementation?

Resources

Demo repository

<https://github.com/cslylla/bubblelux-deepl>

DeepL API documentation

developers.deepl.com/docs/getting-started/quickstart

HTML handling v2

developers.deepl.com/docs/xml-and-html-handling/tag-handling-v2

HTML handling v1

developers.deepl.com/docs/xml-and-html-handling/html

DeepL Python library

github.com/DeepLcom/deepl-python

API client libraries

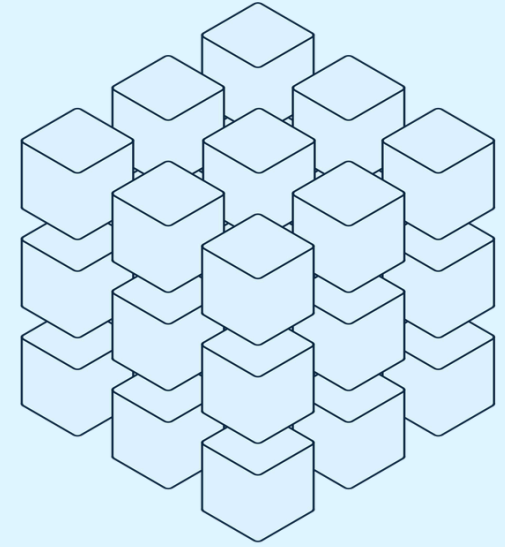
developers.deepl.com/docs/api-reference/client-libraries

Postman collection

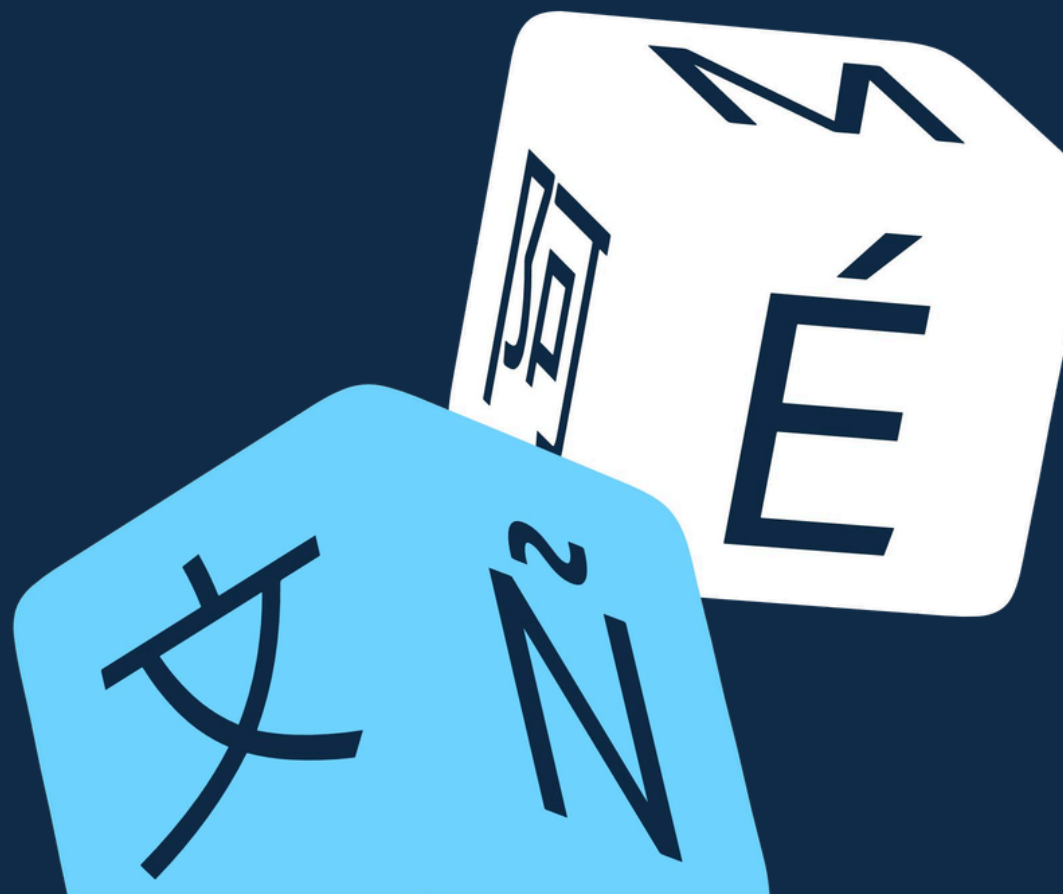
developers.deepl.com/docs/getting-started/test-your-api-requests-with-postman

Free API account

deepl.com/pro-api



Questions?

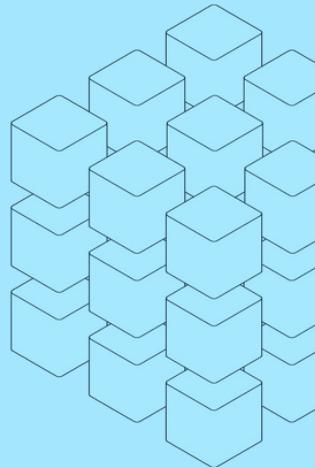


Meet DeepL

Founded in 2017 in Cologne, Germany

We are on a mission to break down language barriers for businesses everywhere

Top 50 fastest
Supercomputer
in the world ¹



¹Top 500

100,000+ customers
and millions of users in
228 global markets



Backed by world-class investors:

atomico^o

ivp

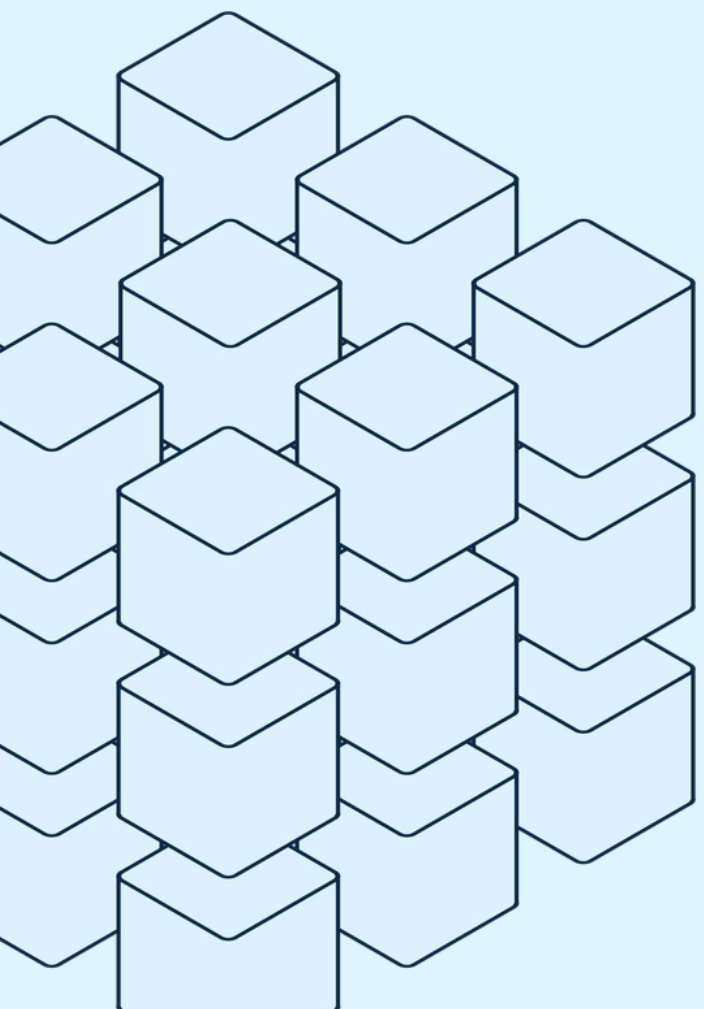
Index
Ventures

ICONIQ | Growth

TEACHERS'
VENTURE GROWTH

WIL
WORLD INNOVATION LAB

DeepL Language AI for Enterprise



Apps

Documents, text, speech



Translator



Write



Voice

Wherever you work



Browser



Microsoft
365



Mobile



Google
Workspace

API



Internal
systems



GTM
systems

Personalization: your enterprise context



Glossary



Your style and tone



Interactivity



Customization

Enterprise-grade security, management, and trust



GDPR, ISO, SOC2



Availability SLAs



Identity mgt



Analytics

Trusted AI-native platform



120 million +
monthly users



Global top 50 fastest
supercomputer



Market-leading
quality



Unique model data,
human-validated



Proprietary LLM
technology

Ensuring your success



Seamless onboarding
and implementation

Manage implementation
effortlessly with our squad of
expert in-house consultants



Best-in-class
technical support

Stay up and running with the
right solutions from our expert
technical support team



Dedicated Customer
Success team

Maximize the value of DeepL
for your business and achieve
your desired ROI



Strong community
of innovators

Discover opportunities that
unlock innovation and drive
a competitive edge